

1. Record architecture decisions

Date: 2026-03-09

Status

Accepted

Context

We need to record the architectural decisions made on this project.

Decision

We will use Architecture Decision Records, as described by Michael Nygard.

Consequences

See Michael Nygard's article, linked above. For a lightweight ADR toolset, see Nat Pryce's `adr-tools`.

2. Orchestration vs Choreography

Date: 2026-03-14

Status

Accepted

Context

The selected application will require different services working in a coordinated manner, therefore we need to define where to use orchestration and where choreography is more suited.

Decision

The ordering and the manufacturing processes will be implemented using orchestration. The customer process will make use of choreography. Both ordering and manufacturing processes need to execute steps with additional logic regarding error and timeout handling. The ordering process also includes manual steps in case an order fails to be fulfilled. The customer process is less critical and can have parts that fail.

Consequences

The orchestration requires the use of a process engine, this increases the complexity of the applications.

3. Operaton as BPMN Engine

Date: 2026-03-14

Status

Accepted

Context

Camunda 7 is EOL, therefore we must choose if we use Operaton or switch to Camunda 8.

Decision

We will implement the system using Operaton as the BPMN process engine.

The system is developed in the context of an IoT application involving physical components, which limits the suitability of cloud-based solutions. Additionally, the selected use case involves only a small number of concurrent process executions. As a result, the scalability benefits offered by cloud-native platforms such as Camunda 8 would not be fully utilized.

Consequences

- The project must be properly configured to use Operaton as the process engine.
- Existing code written for Camunda 7 may require adaptation to ensure compatibility with Operaton.

4. Commands or events

Date: 2026-03-14

Status

Accepted

Context

The order process (orchestrator) must trigger the factory and receive its outcome. For this specific interaction, we need to decide whether to use commands, events, or a mix.

The orchestration boundary is decided in ADR 2.

Decision

The orchestrator sends **commands** to trigger services and services respond with **domain events**.

- The order process sends a **Manufacture Order** command to the factory service.
- The factory service emits domain events to report manufacturing outcome (**Manufacturing complete** or **Manufacturing failure**).
- Manufacturing timeout is not a factory event. It is handled by a 10-minute BPMN timer in the order process, which then emits an error event.

Consequences

- The orchestrator retains explicit control while the factory service remains a smart endpoint, it emits what happened without knowing what the order process does with it.
- Additional consumers can subscribe to factory outcome events without requiring changes in the factory service.
- The **Manufacture Order** command schema is a coupling between the order and factory services. Field changes require coordinated evolution (for example explicit versioning and/or schema registry).
- The correlation ID must be present in the **Manufacture Order** command and echoed in factory outcome events so the order process can correlate replies to the correct process instance.

5. Item color owned by inventory

Date: 2026-03-24

Status

Accepted

Context

The color of an order item can be specified on order. Multiple services need consistent access to this value during processing, e.g. during manufacturing. We had to decide whether color should be sent in the order message or resolved from a shared source.

Decision

Use inventory as the source of truth for item color.

- The order message does **not** contain color.
- The order message contains only:
 - `orderId` (string, Order UUID)
 - `itemType` (enum, name of item to manufacture)
- Services that need color read it from inventory using order/item context.

Consequences

- Color data is managed in one place, avoiding duplication and mismatches across services.
- Order payloads stay smaller and more stable.
- If we look only at exchanged commands, the selected color is not visible.
- Services depend on inventory availability to resolve color.
- Inventory schema/versioning for color must be managed carefully because multiple services rely on it.

6. Dashboard as a Separate Service

Date: 2026-04-04

Status

Accepted

Context

The inventory service initially included a built-in UI for managing the inventory grid. When the requirement arose to show live order progress, we needed to consume Kafka events and push them to the browser in real time. Browsers cannot connect to Kafka directly, so a server-side bridge was required. Adding Kafka consumer and WebSocket logic to the inventory service would mix two unrelated concerns into a service whose sole responsibility is managing stock.

Decision

Extract the dashboard into a dedicated Spring Boot service (**services/dashboard**, port 8100). This service is responsible for:

- Consuming events from Kafka topics **info.v1** and **error.v1**
- Maintaining WebSocket connections with connected browsers
- Bridging Kafka events to the browser in real time
- Serving the frontend UI (inventory grid + order tracker)

The inventory service remains a pure REST API with no UI responsibilities.

Consequences

- The dashboard can be deployed, restarted, or scaled independently of the inventory service.
- A failure in the inventory service does not take down the order status view, and vice versa.
- The browser connects to the dashboard service via WebSocket (**/ws/events**) and polls the inventory service directly for grid state.
- One additional service to run and configure.
- The dashboard has no direct access to inventory state; it relies on the inventory REST API and Kafka events only.

7. Kafka Topic Structure

Date: 2026-04-13

Status

Accepted

Context

The system uses Kafka for asynchronous cross-service communication. We needed a convention for organizing topics that makes ownership and data flow direction clear, and that distinguishes between messages directed at a specific service and messages broadcast to any interested consumer.

Decision

Separate topics by intent into **commands** and **events**:

- **Commands** are issued by an orchestrator to a specific service to trigger work (e.g. `order.manufacture.v1`, `Order` → `Factory`).
- **Events** are published by services to announce outcomes or status, consumed by one or more subscribers (e.g. `order.complete.v1`, `info.v1`, `error.v1`).

Topics follow a `<domain>.<intent>.v1` naming convention, with the version suffix reserved for future schema evolution.

Producers do not use explicit partition keys. Correlation across related messages relies on payload fields (`orderId`, `correlationId`) rather than partition-based routing.

Consequences

- Topic ownership and data flow direction are immediately clear from naming and the command/event distinction.
- Adding new event consumers requires no changes to producers.
- Without partition keying, per-partition ordering guarantees cannot be relied on across related messages; ordering must be handled at the application level.
- The `v1` suffix creates a clear migration path for breaking schema changes (introduce `v2`, run both in parallel, retire `v1`).

8. Kafka Deployment and Consumer Offset Policy

Date: 2026-04-13

Status

Accepted

Context

We needed to decide how Kafka is deployed for the current project scope and how consumers should behave on startup when no committed offset exists.

The system coordinates a physical manufacturing process. Commands on Kafka trigger real-world actions that cannot be safely repeated e.g. replaying an old `order.manufacture.v1` would mean manufacturing an order a second time.

Decision

Run Kafka as a single-node deployment, one process holding both broker and controller roles. Topics use broker defaults (one partition, replication factor one), matching the single-node setup.

All consumers use `auto.offset.reset=latest`. Recovery after downtime is handled through persisted workflow state and the physical state of the factory, not through Kafka replay.

Consequences

- Operational complexity stays low, appropriate for the project scope.
- The broker is a single point of failure; a broker failure takes the messaging layer down entirely. Accepted tradeoff.
- `latest` prevents accidental re-execution of commands after a consumer restart.
- The tradeoff is that messages published while a consumer is down are not seen on reconnect.
- Moving to a multi-broker cluster will require explicit configuration of partition counts, replication factors and likely a revisit of the offset reset policy.

9. Granularity of Factory Service

Date: 2026-04-16

Status

Accepted

Context

The factory domain contains many independent services and functions, all related to manufacturing the order. This decision relates to the granularity of the BPMN process. The process engine is able to model these functions at different levels of abstraction: from the highest-level abstraction of a single service task down to individual robot or sensor actions. The key trade-off here is between the complexity of the model and the flexibility in implementing and extending the process.

Decision

The assembly process is encapsulated into one service task.

The logic of where to pick blocks from, how to move them around, and how to read and interpret sensors is implemented inside one service task. All additional services pertaining to the factory domain are only called by the factory service directly and not exposed to the process engine. With this decision we forfeit flexibility for a simpler Operaton implementation, allowing us to focus on other tasks such as improving the robot service and implementing a custom color sensor.

Consequences

The BPMN process is simplified greatly, and its purpose is limited to providing controlled execution and error handling at a high level. Changing the process requires redeploying the factory service, this is an acceptable trade-off in the context of a physical system where the amount of concurrent operations is limited to one. The assembly logic can be tested directly in the Java application without requiring deployment of the process definition. The extension of the process is limited to developers, a non-technical person is not able to graphically reconfigure the system.

10. Communication Protocol per Integration

Date: 2026-04-24

Status

Accepted

Context

The system integrates very different communication partners:

- Internal business services
- Web-based UI
- Network-connected hardware

A single protocol for all integrations would either add unnecessary complexity for browser and CRUD-style interactions, or create avoidable request/response overhead for streaming hardware signals.

Existing implementation constraints also matter:

- The existing Dobot control is implemented as REST.
- Tinkerforge already publishes sensor data via MQTT.
- Inventory must be usable by backend services and our web-based UI through one stable interface.

Decision

Use a protocol per integration boundary, not one protocol for everything:

- **Inventory API: REST/HTTP**
 - One interface for both backend services and HTML frontend.
 - Simple request/response semantics for reserve, fetch, restore and grid reads.
- **Dobot control: REST/HTTP**
 - New implementation aligned with the existing REST contract.
- **Tinkerforge telemetry: MQTT**
 - Matches existing publisher behavior.
 - Event stream fits pub/sub and decoupled consumers.
- **Color sensor over network: MQTT**
 - Low overhead for frequent sensor messages.
 - Asynchronous publish/subscribe avoids blocking request/reply waits and supports multiple consumers.

Consequences

- Protocol choice matches the nature of each integration and reduces accidental complexity.
- Inventory remains easy to consume from browser code and backend code through one HTTP contract.
- Sensor paths (Tinkerforge and color sensor) become event-oriented and scalable for additional subscribers.

- The system operates multiple protocols (HTTP + MQTT), so operations and monitoring must cover both.

11. Rising-Edge Detection for Sensor Triggers

Date: 2026-05-10

Status

Accepted

Context

Distance sensors publish a continuous stream of readings while a block is in range, potentially at rates of hundreds of messages per second. The downstream consumer (conveyor controller) expects exactly one command per block arrival. Forwarding every raw reading directly would either require consumers to implement their own deduplication, or result in repeated commands being sent to the physical hardware.

Decision

Implement rising-edge detection in `MoveBlockTopology` using a stateful KTable `aggregate`. A trigger event is emitted exactly once when the gap between the previous and current reading exceeds 2 seconds, indicating a new block has appeared after a period of absence. This keeps deduplication inside the stream processing layer and out of the hardware control services.

Consequences

- Each downstream consumer receives exactly one command per block regardless of sensor rate or how long the block lingers in front of the sensor.
- The 2-second gap threshold couples the topology to the physical conveyor speed. If operating conditions change, the threshold must be updated.
- Two blocks arriving within 2 seconds of each other are treated as one arrival. This is acceptable given the physical constraints of the conveyor, where sub-2-second intervals are not achievable at normal operating speed.

12. Wall-Clock Punctuation for Block Detection

Date: 2026-05-10

Status

Accepted

Context

Block detection requires emitting an event once a block has fully passed the sensor, i.e. after a period of inactivity. Kafka Streams session windows and the **Suppress** API both rely on stream time advancing, which only happens when new records arrive. In the current setup, the next event is only emitted when a new block is placed in front of the sensor. This leads to the inventory representation to be out of sync with the physical inventory. With high frequency restocking this would not be an issue, but for small inventory size and individual block restocking operations it is not acceptable.

Decision

Use a custom **BlockInactivityProcessor** with wall-clock punctuation (every 200 ms) instead of a session window. The processor emits a **BlockDetectedEvent** when a sensor key has been quiet for 3 seconds of wall-clock time.

Consequences

- Block detection fires reliably even when the sensor stream is idle, which is essential for the simulator and for low-throughput scenarios.
- The punctuation fires at a fixed interval regardless of load, creating minor CPU overhead when the conveyor is idle.
- The implementation is more verbose than a session window and bypasses the Kafka Streams DSL, requiring manual state store management.

13. JSON Serialization over Avro

Date: 2026-05-26

Status

Accepted

Context

Kafka Streams requires a serialization format for all topics it reads and writes. Avro with a schema registry provides schema evolution guarantees and a compact binary wire format, but requires running and operating a schema registry alongside the Kafka broker.

Decision

Use a custom `JsonSerde<T>` backed by Jackson for all stream processing topics. No schema registry is deployed.

Given the data structures being used are very small, and the prototype nature of this application, setting up Avro would provide limited benefits for a higher initial effort.

Consequences

- The setup remains self-contained: no additional infrastructure is required beyond the Kafka broker.
- Schema evolution is not enforced, producers and consumers must be updated in sync to avoid deserialization failures.
- JSON payloads are larger than Avro binary, which is acceptable for the data volumes produced in this project.